Final Group Interview Practice Problems

Python OOP - Real-Time Business Implementation Challenges

Each group should choose one problem and build a complete Python solution. The solution should show practical thinking, clean class design, validation, error handling, and saving output data into files.

Core concepts expected across the solutions: class, object, methods, multi-class linking, encapsulation, private variables, inheritance, method overriding, polymorphism, abstraction, exception handling, and file handling.

Problem 1: Retail Sales Order and Customer Loyalty System

Business Context

A retail company sells products through an online store. The company wants a small backend system to manage customer orders, apply loyalty benefits, handle payment, update stock, and store order records for later sales analysis.

Scenario

- The system should support different customer types such as regular customers, loyalty customers, and business customers. Each customer type may have different discount rules or benefits.
- Customers should be able to place orders for products only if stock is available and payment can be completed.
- The system should maintain correct wallet/payment behavior and should not reduce product stock when payment fails.
- Every order, whether successful or failed, should be recorded because failed orders are also useful for business analysis.

Expected Implementation Thinking

- Design the required classes by identifying the real-world entities in the problem.
- Use private variables for sensitive values such as wallet balance, product price, stock, payment status, or any value that should not be changed directly from outside.
- Use inheritance and method overriding to handle different customer benefit rules.
- Use polymorphism wherever the same action can behave differently for different customer or payment types.
- Use abstraction so that placing an order is simple from outside, while internal checks are handled inside the system.
- Handle invalid cases such as wrong quantity, unavailable stock, insufficient balance, invalid product price, or invalid customer input.
- Save all order records into a file suitable for later reporting.

Reporting Output Expected

- Total successful orders
- Total failed orders
- Total revenue collected
- Most ordered product
- Failure reason summary

Submission Deliverables

- Python code file or notebook
- Output screenshots
- Saved order file
- Short note explaining class design and business rules handled

Problem 2: Food Delivery Order, Payment, and Restaurant Report System

Business Context

A food delivery company wants a small order management system. Customers order food items from restaurants, pay through different payment options, and the system tracks successful and failed orders for operations reporting.

Scenario

- The system should support food items from multiple restaurants with available quantity and price.
- Customers can pay through different payment modes such as wallet, UPI, card, or cash on delivery. Each payment mode may behave differently.
- Food quantity should reduce only after the order is successful based on quantity availability and payment result.
- The system should store every order record for restaurant-level analysis.

Expected Implementation Thinking

- Identify and create the classes needed for customers, food items, restaurants if needed, payments, orders, and file management.
- Use encapsulation for wallet balance, food price, available quantity, payment result, and order status where needed.
- Use polymorphism for different payment options using a common payment action.
- Use abstraction so that the user can place a food order through one main action while the internal process handles availability, payment, quantity update, and status update.
- Use exception handling for invalid quantity, text input where a number is expected, missing food item, or payment failure.
- Save all order details into a file named appropriately for food order history.

Reporting Output Expected

- Restaurant-wise number of orders
- Successful vs failed orders
- Total revenue by restaurant
- Most failed reason
- Most ordered food item

Submission Deliverables

- Python code file or notebook
- Saved food order history file
- Output showing all test cases
- Short explanation of payment polymorphism and order flow

Problem 3: Movie Ticket Booking and Seat Management System

Business Context

A cinema booking platform wants a system to handle movie ticket bookings. Customers should be able to book seats only when seats are available and payment succeeds. The company also wants booking records for daily performance reports.

Scenario

- The system should manage movies with ticket price and available seats.
- Different customer categories may receive different offers, such as normal customer, student customer, or premium member.
- The system should allow different payment methods with a common payment behavior.
- Seats should reduce only after successful booking and successful payment.
- Failed bookings should also be stored because they help identify payment failures, seat shortage, and invalid booking attempts.

Expected Implementation Thinking

- Design the class structure without hardcoding all logic in one place.
- Protect sensitive values such as wallet balance, ticket price, available seats, and booking status.
- Use inheritance and method overriding for customer category based offer rules.
- Use polymorphism for payment methods or customer offer behavior where suitable.
- Use abstraction so that booking a ticket is handled through a simple booking action while internal steps are hidden.
- Handle invalid seat count, unavailable seats, insufficient payment balance, and wrong input types safely.
- Save all booking records into a file for later analysis.

Reporting Output Expected

- Total tickets booked
- Total booking revenue
- Movie-wise successful bookings
- Movie-wise failed bookings
- Remaining seats report

Submission Deliverables

- Python code file or notebook
- Saved booking file
- Output for successful and failed bookings
- Short note explaining how failed cases are handled

Problem 4: Employee Equipment Request and Inventory Tracking System

Business Context

A company IT team manages laptops, monitors, keyboards, headsets, and other office equipment. Employees request equipment, managers approve special requests, and the system must track inventory usage and rejected requests.

Scenario

- The company has employees from different roles such as intern, full-time employee, manager, and director. Different roles may have different request limits or approval rules.
- Employees can request equipment only if the requested quantity is valid and inventory is available.
- Some expensive equipment may require approval, while basic items can be issued directly.
- Inventory should reduce only when a request is approved and issued.
- Every request should be recorded for audit and inventory analysis.

Expected Implementation Thinking

- Identify the required classes from the business flow and design them clearly.
- Use private variables for inventory quantity, equipment cost, request status, approval status, or other sensitive values.
- Use inheritance and method overriding to handle different employee roles and request rules.
- Use polymorphism where different request types or employee types follow the same action name but different rules.
- Use abstraction so that submitting a request is simple from outside while approval, validation, and inventory update happen internally.
- Use exception handling for invalid quantity, unavailable inventory, invalid employee role, missing approval, or wrong input type.
- Save request history into a file for audit and later reporting.

Reporting Output Expected

- Total approved requests
- Total rejected requests
- Most requested equipment
- Role-wise equipment usage
- Current remaining inventory

Submission Deliverables

- Python code file or notebook
- Saved request history file
- Output for approved and rejected requests
- Short note explaining class responsibilities and validation rules